Does Model Size Matter? A Comparison of Small and Large Language Models for Requirements Classification

Mohammad Amin Zadenoori[1 [0000−0003−4591−153X]], Vincenzo De Martino[2 [0000−0003−1485−4560]], Jacek Dąbrowski[3 [0000−0003−3392−0690]], Xavier Franch[2 [0000−0001−9733−8830]], and Alessio Ferrari[4,5 [0000−0002−0636−5663]]

[1] University of Padova, Italy
amin.zadenoori@unipd.it
[2] Universitat Politècnica de Catalunya, Spain
{vincenzo.de.martino, xavier.franch}@upc.edu
[3] Lero, the Research Ireland Centre for Software, University of Limerick, Ireland
jacek.dabrowski@lero.ie
[4] University College Dublin (UCD), Ireland
alessio.ferrari@ucd.ie
[5] CNR-ISTI, Via G. Moruzzi 1, 56124 Pisa, Italy
alessio.ferrari@isti.cnr.it

Abstract. [Context:] Large language models (LLMs) show notable results in natural language processing (NLP) tasks for requirements engineering (RE). However, their use is compromised by high computational cost, data-sharing risks, and dependence on external services. In contrast, open-source small language models (SLMs) offer a lightweight, locally deployable alternative, yet evidence comparing their effectiveness with LLMs in RE is limited. **[Goal:]** This study investigates whether model size significantly affects performance in requirements classification by benchmarking SLMs against LLMs. **[Method:]** We conducted an empirical comparison of eight language models (five SLMs and three LLMs) across three widely used datasets. Performance was evaluated using precision, recall, and F1 score, and analyzed using nonparametric statistical tests. **[Results:]** LLMs showed a slight average advantage (2% higher F1 score), though this difference was not statistically significant. SLMs achieved comparable performance across all datasets and even outperformed LLMs in recall on one of the three datasets. Further statistical analysis indicated that dataset characteristics substantially and significantly affected performance, whereas model type (SLM vs. LLM) did not. **[Conclusions:]** Our findings suggest that, for the evaluated datasets and prompting configuration, the selected SLMs achieve performance close to that of the selected LLMs, indicating that model size plays a limited role in this specific setting. Considering their advantages in privacy, local deployment, and resource efficiency, SLMs constitute a viable and cost-effective alternative to LLMs for RE tasks. This work presents the first benchmark study comparing SLM and LLM for requirements classification.

Keywords: Artificial Intelligence for Software Engineering · Requirement Engineering · Language Models · Requirements Classification

1 Introduction

Requirements classification is a critical task in requirements engineering (RE), which involves organizing requirements into different categories such as functional and non-functional requirements, or more fine-grained quality attributes (e.g., security, performance or reliability) [30]. Such classification supports efficient requirements analysis and management [2], and it is practically relevant for the whole software process, as it facilitates downstream RE activities such as requirements allocation to teams [7] and requirements tracing [1]. As projects grow larger, the number of requirements can reach thousands, making manual classification labor-intensive, subjective, and error-prone.

This has motivated software engineering (SE) and RE research to automate requirements classification task using natural language processing (NLP) techniques [30]. Recent advances in large language models (LLMs), such as GPT and Claude, have transformed NLP, achieving state-of-the-art results in text understanding, classification, and generation. Consequently, the RE community is increasingly exploring their use for RE tasks, including automated requirements classification [18,28,27].

Despite their unprecedented capabilities, LLMs raise concerns about privacy, security, reproducibility, and sustainability. They are typically closed-source and cloud-hosted, which increases the risk of data exposure, as company requirements constitute confidential assets [17] and the cloud operates as an external service. Moreover, the proprietary nature of LLMs restricts their adaptation to RE and company-specific needs and limits their practical use in industry. The carbon footprint of these models raises further ethical concerns, particularly regarding their environmental impact.

In contrast, open-source small language models (SLMs), typically comprising up to a few billion parameters rather than the tens to hundreds of billions found in LLMs, offer a viable lightweight alternative, as they support local deployment and secure processing of sensitive data. Their local deployment also lowers costs and enables their customization and fine-tuning [25]. Furthermore, SLMs offer advantages in terms of reduced energy consumption [13].

While some studies have investigated the performance of SLMs for requirements classification [3,28,24], and LLMs [8,18], it remains unclear how SLMs perform relative to LLMs on these tasks. The lack of direct comparative evidence limits our understanding of the respective strengths, weaknesses, and trade-offs of these technologies in classifying requirements.

To address this gap, this paper conducts an empirical comparison of eight popular language models (LMs) for the requirements classification task. We evaluate five SLMs with 7-8 billion parameters (*Qwen2-7B*, *Falcon-7B*, *Granite-3.2-8B*, *Ministral-8B*, and *Llama-3-8B*) and three LLMs (*GPT-5*, *Grok-4*, and *Claude-4*). LMs were selected through an initial qualitative screening using the

Hugging Face chat interface. Rather than relying solely on rankings such as the Hugging Face Open LLM Leaderboard[6], which emphasize general-purpose benchmarks and evolve rapidly, we prioritized (i) strong performance on text classification tasks, (ii) diversity across model families, and (iii) availability of open-weight models with permissive licenses. LMs were evaluated on three open-source datasets for RE tasks: PROMISE [9], a re-classification of PROMISE (referred to as PROMISE Reclass) [10], and SecReq [21].

The main contributions of this paper are: (i) an empirical comparison of five SLMs and three LLMs for requirements classification; (ii) empirical evidence that SLMs achieve performance comparable to LLMs despite being significantly smaller; and (iii) insights into the impact of dataset characteristics and model size on performance, as well as their implications for RE research. To the best of our knowledge, this is the first systematic study directly comparing SLMs and LLMs for requirements classification.

Data availability statement: To ensure verifiability and replicability, we provide a replication package with all raw data, the dataset, prompts, and analysis scripts [5].

2 Related Works

Among NLP applications in RE, requirements classification is the most extensively investigated task [31]. Several techniques have been explored, including (1) classic machine learning approaches, e.g., Support Vector Machines (SVMs) and Naive Bayes (NB), (2) deep learning techniques, e.g., Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) (3) transfer learning approaches, e.g. BERT, and, more recently, (4) also LLMs and SLMs.

Machine Learning Approaches. Early research from Cleland-Huang et al. [9] focused on distinguishing between functional requirements (FRs) and non-functional requirements (NFRs), and identifying different NFR classes, by means of a probability function similar to NB. This study also introduced the PROMISE dataset, which has been later widely used by the RE community [22,11,19,2]. On the same task and dataset, Kurtanović ad Maalej [22] experimented with SVMs, showing the superiority of these machine learning models with respect to the previously experimented techniques, with a performance increase of over 10%. The study was reproduced by Dalpiaz et al. [11], who also re-classified the PROMISE dataset, based on functional and quality classes. This PROMISE Reclass dataset treats these categories as non-mutually exclusive, in line with a more contemporary vision of requirements categorization [15]. In addition to NFR classification tasks, Knauss et al. [20] performed a study on security requirements classification, using a NB classifier to identify security-relevant requirements on three industrial datasets (the public SeqReq dataset).

[6] https://huggingface.co/spaces/open-llm-leaderboard

Deep Learning Techniques. An early deep learning approach to requirements classification was proposed by Winkler and Vogelsang [26], who implemented a CNN classifier using word2vec-based embeddings and max-pooling. The model was trained on a dataset constructed from 89 industrial requirements documents. CNNs were used also by Dekhtyar and Fong [14] and by Baker *et al.* [6], who leveraged again the PROMISE dataset, as well as SecReq. These techniques did not substantially improve the performance with respect to SVMs, showing that more complexity does not necessarily mean better effectiveness.

Transfer Learning. With the advent of transfer learning, further experiments were conducted. Hey et al. [19] introduced NoRBERT, an approach that fine-tunes pre-trained $\text{BERT}_{\text{base}}$ and $\text{BERT}_{\text{large}}$ models for binary and multi-class requirements classification using PROMISE, PROMISE-reclass, and SeqReq. The approach outperformed all the previous baselines, achieving an F1-score of over 90%, which, to our knowledge, are still state-of-the-art. Fine-tuning, however implies the use of annotated requirements samples, which are often scarce in RE [17]. To address this problem, Alhoshan et al. [2] presented the first study where an embedding-based zero-shot learning (ZSL) approach was applied to nine well-known LMs, including BERT, without fine-tuning or training. Although lower performance were obtained with respect to previous work, the study showed that, even in the absence of training data, LMs are still competitive, achieving an F1-score of 82%. BERT and similar LMs have been used also in industrial studies, focused on requirements classification for team allocation [7] and to support traceability [1].

LLMs and SLMs. LLMs and SLMs are appealing for requirements classification, as, similarly to zero-shot learning, they do not need annotated requirements data. Experiments with LLMs have been performed, among others, by Binkonain et al.[8], who experimented with different prompts and different LLMs, e.g., Deepseek, ChatGPT, on PROMISE and SecReq, showing that LLMs can achieve better performance than the state-of-the-art NorBERT model by Hey et al. [19]. Conversely, on the same datasets, and using an inference-based approach, SLMs have demonstrated weaker performance [4]. Other works on SLMs using these datasets have focused on automatic prompt engineering [29] and green prompting [13], and also showed that SLMs do not appear to improve over NorBERT.

> 💡 ***Research Gap and Contribution.*** *Extensive research has been performed on requirements classification, often suggesting the dominance of LLMs over SLMs. However, to the best of our knowledge, this assumption has not been systematically validated. To bridge this gap, we conduct a benchmark study on established datasets, demonstrating that LLMs and SLMs achieve comparable performance, with SLMs offering a more cost-effective alternative. Our research approach aims to deepen understanding of automated requirements classification, foster stronger connections among AI, RE, and SE, and provide insights for future research.*

3 Methodology

This section describes the study design employed to evaluate and compare SLMs and LLMs for requirements classification, guided by the following research question (RQ):

> **RQ:** *How effective are SLMs compared to LLMs in requirements classification tasks?*

To answer our RQ, we evaluated the LMs on a human-annotated dataset, comparing their predictions against ground-truth labels using precision, recall, and F1 score. We then applied statistical tests to assess whether the observed performance differences were statistically significant.

We overall followed the PRIMES framework [12]. We selected this framework as it provides a structured approach for the systematic evaluation of LMs. The instantiation of the framework for our empirical study consisted of six major elements: (i) dataset, (ii) LMs, (iii) prompt engineering, (iv) experimental parameters and setup, (v) evaluation procedure, and (vi) statistical analysis.

Datasets. We considered three publicly available datasets widely used in the RE community: PROMISE [9], Dalpiaz et al.'s PROMISE variant that we call PROMISE Reclass [10], and SecReq [21]. These datasets were chosen because they contain manually annotated requirements suitable for evaluating classification tasks and represent different categorization schemas, thus enhancing the generalizability of our results.

The PROMISE dataset [9] includes 625 requirements, comprising 255 Functional Requirements (FR) and 370 Non-Functional Requirements (NFR). PROMISE Reclass [10] contains the same 625 requirements, re-annotated according to two alternative classification schemes: (i) FR vs. NFR and (ii) Quality Requirements (QR) vs. Non-Quality Requirements. Unlike the original PROMISE dataset, labels in PROMISE Reclass are not mutually exclusive, meaning a requirement can simultaneously belong to multiple categories. Finally, SecReq [21] comprises 510 requirements, including 187 security-related and 323 non-security requirements.

Models. We selected eight language models, comprising five open-source SLMs (7B–8B parameters, locally executable) and three proprietary LLMs. The SLMs include Qwen2-7B-Instruct, Falcon-7B-Instruct, Granite-3.2-8B-Instruct, Ministral-8B-Instruct-2410, and Meta-Llama-3-8B-Instruct. These models are openly available, support local deployment, and offer realistic solutions for researchers and practitioners with limited computational resources. We focus on 7B–8B SLMs as they represent a practical trade-off between performance and local deployability, aligning with realistic constraints faced by researchers and practitioners. Future work will extend this analysis to smaller models and explore comparisons within the same model families to better isolate the impact of LMs' size.

The LLMs include GPT-5, xAI Grok-4, and Claude-4. These systems are proprietary and accessed via commercial APIs. Their exact parameter counts are not publicly disclosed by their providers. However, based on available technical

reports, public statements, and comparisons with prior generations, they are widely understood to exceed 100B parameters. We therefore treat them as representatives of large-scale, closed-source systems. LM's selection was guided by their strong performance in widely used public benchmarks, including the Hugging Face Open LLM Leaderboard [7], ensuring the inclusion of competitive and state-of-the-art models across different scales and deployment settings.

Prompting Strategy. To evaluate the LMs' ability to perform this task, we adopted a prompting approach that combines Chain-of-Thought (CoT) and Few-Shot prompting; we include CoT reasoning steps, and Few-Shot labeled examples in a single prompt template, providing examples directly within the prompt to guide the LLM. We included four labeled examples per target class in the prompt to guide the models' learning of the classification criteria. We selected this approach based on prior work [29], which identified it as the most effective strategy. It outperformed several alternative prompting techniques for requirements classification. We grounded our requirements category definitions in the RE literature [23,16] to ensure conceptual validity and consistency with established domain knowledge. We used the same prompt structure, examples, and instructions for all models to ensure a fair and reproducible comparison. We used a single fixed prompt without iteration, and the same version was applied across all experiments. The complete prompt template is provided in the replication package to support transparency and reproducibility.

Experimental Parameters and Setup. The LMs were executed using their default configurations. We set the temperature to 0 and fixed the random seed to reduce non-determinism and enhance reproducibility.

The SLMs were deployed on a Linux 6.14 server equipped with an Intel i9-13900K CPU, 128 GB RAM, and an NVIDIA RTX 4090 GPU. All experiments were implemented in Python 3.12. Proprietary LLMs were accessed via their respective commercial APIs. No hyperparameter tuning was performed, allowing us to isolate the effect of the prompting strategy and ensure a consistent and fair comparison across LMs.

Statistical Analysis. To evaluate the effects of model type and dataset on classification performance, we conduct a factorial non-parametric analysis using the Scheirer–Ray–Hare test, a rank-based alternative to two-way ANOVA suitable for non-normal data. The dependent variable is the F1 score. Two fixed factors are considered: (i) *Model Type* (SLM vs. LLM) and (ii) *Dataset* (PROMISE, PROMISE Reclass, SecReq).

We formulated the following null hypotheses:

- H_{0A}: There is no main effect of Model Type on F1 score.
- H_{0B}: There is no main effect of Dataset on F1 score.
- H_{0C}: There is no interaction effect between Model Type and Dataset.

Normality was assessed using the Shapiro–Wilk test and was not satisfied; therefore, a non-parametric approach was adopted.

When significant effects were detected, post-hoc pairwise comparisons were performed using Mann–Whitney U tests with Bonferroni correction.

[7] `https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard`

Table 1. Results across all datasets using CoT ∪ Few-shot.

Language	PROMISE			PROMISE Reclass			SecReq		
Model	P	R	F1	P	R	F1	P	R	F1
Qwen2-7B	0.86	0.80	0.83	0.55	**0.96**	0.70	0.83	0.90	0.86
Falcon3-7B	0.86	0.80	0.83	0.58	**0.96**	0.72	0.79	0.90	0.84
Granite-3.2	0.84	0.82	0.83	0.62	0.87	0.72	0.84	0.90	0.87
Ministral-8B	0.84	0.77	0.80	0.64	0.89	0.75	0.83	0.88	0.85
Llama-3-8B	0.84	0.77	0.80	0.70	0.87	0.78	0.86	0.91	0.88
SLMs (Mean)	0.85	0.79	0.82	0.62	0.91	0.73	0.83	0.90	0.86
Grok-4	**0.88**	**0.83**	**0.85**	0.60	0.82	0.69	0.84	0.89	0.86
GPT-5	0.85	0.81	0.83	0.68	0.88	0.77	0.85	0.90	0.87
Claude-4	0.85	0.80	0.82	**0.72**	0.90	**0.80**	**0.87**	**0.92**	**0.89**
LLMs (Mean)	0.86	0.81	0.83	0.67	0.87	0.75	0.85	0.90	0.88

4 Results

This section presents the results of our investigation, analyzing the performance of the evaluated models across the selected datasets to address our RQ. Table 1 reports classification performance per dataset, including average metrics, considering precision (P), recall (R), and F1.

Overall Performance. Across the three datasets, LLMs obtain slightly higher mean F1 scores than SLMs; however, the differences remain limited in magnitude. On the **PROMISE** dataset, precision ranges from 0.84 to 0.88 and recall from 0.77 to 0.83 across all models, indicating relatively limited variability. F1 scores range between 0.80 and 0.85. Grok-4 achieves the highest F1 score (0.85), while Ministral-8B and Llama-3-8B report the lowest (0.80). Qwen2-7B, Falcon3-7B, and Granite-3.2-8B reach 0.83, resulting in a maximum F1 gap of 0.05 across models. The mean F1 score is 0.82 for SLMs and 0.83 for LLMs.

On the **PROMISE Reclass** dataset, performance variability increases. Precision varies between 0.55 and 0.72, and recall between 0.82 and 0.96. F1 scores range from 0.69 to 0.80. Claude-4 attains the highest F1 score (0.80), whereas Grok-4 records the lowest (0.69). Qwen2-7B and Falcon3-7B achieve the highest recall (0.96), while Qwen2-7B reports the lowest precision (0.55). The mean F1 score is 0.73 for SLMs and 0.75 for LLMs.

On the **SecReq** dataset, all LMs demonstrate consistently high performance. Precision ranges from 0.79 to 0.87 and recall from 0.88 to 0.92. F1 scores vary between 0.84 and 0.89. Claude-4 achieves the highest F1 score (0.89), while Falcon3-7B reports the lowest (0.84). The difference between the best and worst F1 scores is 0.05. The mean F1 score is 0.86 for SLMs and 0.88 for LLMs.

Overall, while individual LLMs achieve the highest F1 scores in each dataset, SLMs demonstrate comparable results and, in some cases, higher recall values.

Hypothesis Testing Results. Table 2 reports the results of the Scheirer–Ray–Hare test evaluating the effects of Model Type, Dataset, and the interaction on F1.

Model Type Effect. The analysis does not reveal a statistically significant main effect of Model Type on F1 score ($H = 1.09$, $p = 0.296$, $\eta^2_H = 0.04$). The effect size is small, indicating that differences between SLMs and LLMs account for a limited proportion of the observed variance in performance.

Table 2. Scheirer–Ray–Hare test results evaluating the effects of Model Type and Dataset on F1 score.

Factor	H	p-value	η^2_H	Decision
Model Type	1.09	0.296	0.04	Fail to Reject H_{0A}
Dataset	17.53	<0.001	0.63	Reject H_{0B}
Model Type × Dataset	0.47	0.790	0.001	Fail to Reject H_{0C}

Dataset Effect. A statistically significant main effect of Dataset is observed ($H = 17.53$, $p < 0.001$, $\eta^2_H = 0.63$), corresponding to a large effect size. This indicates that dataset characteristics explain a substantial portion of the variability in F1 scores. Median F1 values follow a consistent ordering across models: PROMISE Reclass (0.730), PROMISE (0.805), and SecReq (0.865).

Interaction Effect. No statistically significant interaction between Model Type and Dataset is detected ($H = 0.47$, $p = 0.790$, $\eta^2_H = 0.001$). Our result suggests that the dataset's influence on performance does not differ between SLMs and LLMs.

Post-hoc Analysis. Pairwise comparisons between SLMs and LLMs (Model Type Effect) within each dataset do not yield statistically significant differences (PROMISE: $p = 0.134$; PROMISE Reclass: $p = 0.764$; SecReq: $p = 0.291$). Bonferroni-corrected pairwise comparisons between datasets (Dataset Effect) reveal statistically significant differences across all dataset pairs (PROMISE Reclass vs. PROMISE: $p = 0.0084$; PROMISE Reclass vs. SecReq: $p = 0.0009$; PROMISE vs. SecReq: $p = 0.0031$).

> 👉 **Answer to RQ:** SLMs are generally as effective as LLMs in classifying requirements. While LLMs tend to achieve slightly higher average F1 scores, this difference is not statistically significant. Instead, performance varies significantly across different datasets, suggesting that the characteristics of the datasets have a more substantial impact than the scale of the models. Overall, SLMs demonstrate effectiveness that is comparable to LLMs in the evaluated context.

5 Threats to Validity

Construct Validity. Performance was evaluated using precision, recall, and F1-score, with F1 selected as the dependent variable for statistical testing because it balances false positives and false negatives and is standard in requirements classification research. However, different application contexts may require alternative β values in the F_β measure to prioritize precision or recall. Identifying such context-specific configurations requires further investigation and was beyond the scope of this study. Another construct-related concern involves model size categorization. Exact parameter counts are available for open-source SLMs

but not publicly disclosed for proprietary LLMs. We therefore rely on publicly available technical documentation and comparative evidence to classify them as large-scale models. Although precise parameter counts are unavailable, the distinction reflects meaningful architectural and deployment differences (local vs. API-based systems). Our selection of SLMs focuses on 7B–8B models, which represent a practical trade-off between performance and local deployability. However, this choice may limit the generalizability of our findings to smaller models. Future work will extend the analysis to include smaller SLMs and models from the same families to better isolate the impact of model size.

Internal Validity. The same prompt was applied across all models to ensure consistency; however, relying on a single prompt, even if informed by prior studies [29], may affect internal validity since model performance could depend on prompt phrasing [3]. In future work, we plan to extend the number of prompting techniques to improve the generalizability of our findings. Internal validity is also affected by the variability of model output. To address this, we first set the temperature to 0 and fixed the random seed to obtain deterministic results. Then, we repeated each run three times and performed majority voting. A further threat concerns potential data leakage, as widely used benchmark datasets may be included in the pre-training corpora of the evaluated LMs, potentially inflating performance. While we acknowledge this risk, we do not quantify its impact in the current study, which could represent a limitation. Future work could address this by conducting overlap analyses (e.g., n-gram matching), membership inference tests, or controlled evaluations using paraphrased or held-out data to better distinguish memorization from generalization, or by using new datasets that have not been used to train LMs.

External Validity. This study focuses on binary requirements classification using three publicly available datasets. Although these datasets differ in labeling schemas and structural characteristics, the findings may not generalize to more complex settings, such as multi-class classification, traceability, or generative RE tasks. The statistically significant dataset effect observed in our analysis suggests that performance is strongly influenced by dataset properties. In this context, we observed a limited impact of model size; however, this finding is specific to the binary classification setup and should not be generalized to other RE tasks without further validation. Future work should evaluate these models on larger, noisier, and industrial datasets to assess real-world applicability and determine whether the observed trends hold beyond controlled benchmark settings.

Conclusion Validity. We performed statistical tests to check our conclusions. However, the limited sample size might have led to Type II errors. To avoid multiple comparison issues across dependent variables, we used only the F1 metric for statistical analysis. However, as F1 may not capture all performance nuances, we also report precision and recall for specific cases in Table 1.

6 Discussion and Implications

Our study highlights several implications for the RE and SE community and practitioners focusing on the use of LMs for RE.

6.1 Effectiveness of SLMs over LLMs in Requirements Classification and Comparison with other Contributions

Across all three datasets, SLMs perform on par with LLMs under CoT ∪ Few-shot prompting, with only marginal differences in F1 and occasional superiority on specific metrics. For instance, Qwen2-7B and Falcon3-7B achieve the highest recall (0.96) on PROMISE Reclass, and Granite-3.2 matches or exceeds several LLMs on SecReq. These results suggest that, within this experimental setting, performance is less driven by model scale and more influenced by how effectively the prompt structures the reasoning process. In particular, the consistently high recall observed for SLMs indicates that smaller models can benefit substantially from explicit reasoning scaffolds, which help reduce ambiguity and enforce clearer decision boundaries during classification.

However, this interpretation must be contextualized. Our findings do not imply that model size is universally negligible, but rather that its impact appears limited under a controlled prompting configuration and for this specific task. Given the known sensitivity of LMs to prompt design, it is plausible that different prompting strategies (e.g., zero-shot, persona-based, or alternative reasoning structures) could amplify or reduce the observed differences between SLMs and LLMs. As such, our results should be interpreted as evidence of comparability under a specific, structured prompting setup, rather than a general statement about model size across all conditions.

When compared to prior generative and non-generative baselines, both SLMs and LLMs show substantial gains: Llama in Alhoshan et al. [3] reports a weighted F1 of 0.6276 for FR vs. NFR, whereas our SLMs reach 0.82 on PROMISE and 0.73 on PROMISE Reclass, and LLMs reach 0.83 and 0.75, respectively, on the same two datasets. Persona-based prompting in Binkhonain et al. [8] yields a slightly higher macro-F1 (0.92 for Gemini-3FSP), but relies on persona engineering and multiple prompt variants, while our single structured prompt achieves comparable results; notably, SLMs reach a mean F1 of 0.82 on PROMISE. Although supervised models such as NoRBERT [19] still obtain the highest scores (0.90–0.95), our SLMs and LLMs achieve 0.82–0.88 *without training data*, underscoring the value of inference-only classification when labeled requirements are scarce. Overall, these findings show that SLMs equipped with structured reasoning prompts deliver performance competitive with LLMs and superior to prior baselines, suggesting that model scale matters less than prompting strategy and positioning SLMs as a cost-efficient alternative for high-quality requirements classification.

6.2 Analysis of Cross-Model Misclassifications

One essential study is needed to understand which elements within the text mislead LMs, given their complex behavior. To this end, for each of the three

datasets, we identify one sample that is misclassified by all eight LMs—a deliberately small number, but sufficient to provide a brief illustration—and examine its characteristics, focusing on why it is vague or misleading.

From the **PROMISE NFR** dataset, the misclassified sample states: "If projected, the data must be readable. On a 10x10 projection screen, 90% of viewers must be able to read Event/Activity data from a viewing distance of 30'." The ground truth identifies this as a non-functional requirement, yet all LMs predicted it to be functional. However, it is unclear whether the functionality should enable this or the quality should define it. The numeric details ("90%", "10x10", "30'") resemble functional metrics, which likely caused all five models to over-index on numbers and misclassify the requirement. The lack of explicit non-functional keywords such as "quality," "usability," or "readability attribute" further adds to this ambiguity.

From the **PROMISE Reclass** dataset, the misclassified sample reads: "The product shall be available during normal business hours. As long as the user has access to the client PC the system will be available 99% of the time during the first six months of operation." The ground truth is that this is a non-functional quality requirement (specifically availability), yet all LMs predicted it to be a functional quality requirement. This sample is misleading because the phrase "the system will be available 99% of the time" contains a concrete numeric threshold (99%) and a temporal bound ("first six months"), which LLMs commonly associate with functional requirements that specify exact behaviors. However, the core concern is availability, a non-functional quality attribute. The mention of "user has access to the client PC" further misleads models into thinking access control (often functional) is involved, obscuring the underlying quality-of-service nature.

From the **SecReq dataset**, the misclassified sample is: "On indication received at the CNG of a resource allocation expiry the CNG shall delete all residual data associated with the invocation of the resource." The ground truth is security, yet all LMs predicted non-security (e.g., resource management or data lifecycle). This sample is misleading because the action "delete all residual data" sounds security-relevant, yet the trigger "resource allocation expiry" is a routine lifecycle event in telecommunications. The complete absence of security keywords such as "unauthorized," "breach," "confidential," or "integrity" causes LMs to rely on surface patterns from training, where similar expiry-triggered deletions are almost always labeled non-security. Additionally, the vague actor "the CNG" and the abstract "invocation of the resource" provide no security boundary context, confirming all LMs in their incorrect prediction.

Across all three datasets, the samples misclassified by every LM share common characteristics: they contain concrete numeric or temporal metrics that mimic functional specifications; they lack domain-specific keywords that would signal their true non-functional or security category; and they rely on underspecified entities or ambiguous boundaries between different conditions. These properties misled all LMs, revealing a fundamental limitation in handling linguistic vagueness and contextual misdirection. In future work, it will be essential to move beyond analyzing just one misclassified sample per LM, as relying on a single example

per model offers insufficient evidence. Instead, future studies should include all misclassified samples from all LMs to ensure a more robust and comprehensive analysis.

6.3 Implications for Research

Generality across RE and SE tasks. The strong performance observed in requirements classification raises the question of whether similar gains would extend to other RE and SE tasks, such as traceability, ambiguity detection, requirements-to-model generation, or bug detection. However, requirements classification is a relatively constrained task with well-defined output categories. Tasks involving generation or more complex reasoning may exhibit different behavior, particularly with respect to model scale. Therefore, further evaluations across diverse tasks are needed to assess whether the observed comparability between SLMs and LLMs holds beyond this specific setting.

Prompt sensitivity and confounding variables. Although we used a single prompt derived from prior literature, existing studies highlight the strong sensitivity of language models to prompt design. For example, Alhoshan et al. [3] report substantial performance variation across prompting strategies, while Binkhonain et al. [8] show that carefully engineered prompts can even outperform supervised models. As a consequence, our findings should be interpreted as specific to the chosen CoT ∪ Few-shot configuration. Moreover, additional confounding variables—such as requirement format (e.g., user stories vs. "shall" statements), domain specificity, ambiguity, and model familiarity with the domain—may influence performance. These factors were not explicitly controlled in our study and should be investigated in future work to better isolate the effects of model size.

Dataset characteristics as a determining factor. Our results indicate that performance varies significantly across datasets, suggesting that dataset properties may play a major role. For instance, datasets with clearer class boundaries and less overlap between functional and non-functional requirements (e.g., SecReq and PROMISE) appear to yield higher performance compared to more ambiguous datasets (e.g., PROMISE-Reclass). However, this interpretation remains exploratory, as the dataset characteristics were not explicitly quantified or controlled for. Future work should aim to analyze factors such as linguistic complexity, class separability, and domain specificity to better understand their impact on model performance.

6.4 Implications for Practitioners

SLMs as a cost-effective alternative to LLMs. Our results show that SLMs achieve performance close to LLMs under a structured prompting strategy, particularly on PROMISE and SecReq. This suggests that organizations may adopt SLMs for requirements classification without fine-tuning or labeled data, potentially reducing computational and financial costs. However, this conclusion is contingent on the use of effective prompting and on tasks with characteristics similar to those evaluated in this study.

Supervised models remain the strongest option when data are available. Despite the strong performance of SLMs and LLMs, supervised models such as NoRBERT still achieve the highest accuracy (F1 $\approx$ 0.90–0.95). When labeled data are available, and high accuracy is critical, these models remain the preferred choice.

Importance of requirements writing. Since dataset characteristics and requirements classes can influence performance, organizations should aim to write requirements that are atomic, unambiguous, and clearly associated with a single class. This applies not only to functional decomposition but also to non-functional attributes. Insights from research on dataset characteristics can guide practitioners in writing requirements that facilitate high-accuracy automated classification. However, further empirical validation is needed to quantify this effect.

7 Conclusions and Future Work

In this paper, we aimed at understanding whether SLMs can represent a viable alternative to proprietary LLMs for requirements classification. Our work provided the following contributions:

1. An empirical comparison of five open-source SLMs and three proprietary LLMs across three public RE datasets;
2. Empirical evidence showing that SLMs achieve performance comparable to LLMs, with only a marginal (2% F1-score) and statistically non-significant difference;
3. A public replication package [5] which may be exploited by researchers to either replicate our work or build on top of our findings.

Overall, our findings indicate that, within the scope of our datasets and experimental setup, **increasing model size does not lead to substantial gains in classification accuracy**. Instead, the practical characteristics of each model family become more relevant when considering deployment. In our setting, SLMs achieved accuracy comparable to LLMs while remaining lightweight enough for local execution, making them a practical option for scenarios where requirements data cannot be processed through external cloud services.

Our research agenda will focus on extending this work beyond pure classification tasks. First, we aim to investigate explainability mechanisms that generate structured rationales for model predictions. In RE contexts, explanations are essential to support traceability, auditing, and informed decision-making, thereby reducing the risk of blindly relying on automated outputs.

Second, we plan to evaluate SLMs and LLMs across a broader spectrum of generative RE tasks, including traceability recovery, artifact generation, and requirements elicitation. These tasks may better expose differences among model families and enable the design of hybrid RE pipelines in which SLMs and LLMs are selectively deployed based on task complexity, privacy constraints, and computational resources.

Third, we will assess energy consumption and inference execution time in RE-specific scenarios. While LLMs may offer slight accuracy improvements, their environmental footprint and infrastructural costs remain critical concerns.

Future studies will quantify performance–efficiency trade-offs across different deployments to better inform sustainable AI adoption in RE.

Finally, we envision developing an adaptive tool that supports automatic requirements classification and benchmarking. Such a tool would enable dynamic selection between locally open-source SLMs and proprietary LLMs, adapt prompting strategies to contextual constraints, and foster reproducibility and transparency in RE workflows.

Acknowledgements This work was supported in part by project EOSS6-0000000644 from the Chan Zuckerberg Initiative and by the Grant PID2024-156019OB-I00 funded by MICIU/AEI/10.13039/501100011033 by ERDF, EU. Grok-4 (https://x.ai/grok) was used to improve the readability, and the authors remain fully responsible for the content.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Abdeen, W., Unterkalmsteiner, M., Wnuk, K., Ferrari, A., Chatzipetrou, P.: Language Models to Support Multi-Label Classification of Industrial Data . In: 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). pp. 45–55. IEEE Computer Society, Los Alamitos, CA, USA (Mar 2025). https://doi.org/10.1109/SANER64311.2025.00013, https://doi.ieeecomputersociety.org/10.1109/SANER64311.2025.00013
2. Alhoshan, W., Ferrari, A., Zhao, L.: Zero-shot learning for requirements classification: An exploratory study. Information and Software Technology **159**, 107202 (2023)
3. Alhoshan, W., Ferrari, A., Zhao, L.: How effective are generative large language models in performing requirements classification? (2025), https://arxiv.org/abs/2504.16768
4. Alhoshan, W., Zhao, L., FERRARI, A.: How effective are generative large language models (llms) in performing requirements classification? assessing the impact of llms, prompts, datasets, and tasks – replication package (Jun 2025). https://doi.org/10.5281/zenodo.15665846, https://doi.org/10.5281/zenodo.15665846
5. authors, A.: Comparison of small and large language models for requirements classification (Apr 2026), https://anonymous.4open.science/r/A-Comparison-of-Small-and-Large-Language-Models-for-Requirements-Classification-2809/README.MD
6. Baker, C., Deng, L., Chakraborty, S., Dehlinger, J.: Automatic multi-class non-functional software requirements classification using neural networks. In: 2019 IEEE 43rd annual computer software and applications conference (COMPSAC). vol. 2, pp. 610–615. IEEE (2019)
7. Bashir, S., Abbas, M., Ferrari, A., Saadatmand, M., Lindberg, P.: Requirements classification for smart allocation: A case study in the railway industry. In: 2023 IEEE 31st International Requirements Engineering Conference (RE). pp. 201–211. IEEE (2023)
8. Binkhonain, M., Alfayez, R.: Are prompts all you need? evaluating prompt-based large language models (llm) s for software requirements classification. Requirements Engineering pp. 1–21 (2025)

9. Cleland-Huang, J., Settimi, R., Zou, X., Solc, P.: Automated classification of non-functional requirements. Requirements engineering **12**(2), 103–120 (2007)
10. Dalpiaz, F., Dell'Anna, D., Aydemir, F., Cevikol, S.: Requirements classification with interpretable machine learning and dependency parsing. pp. 142–152 (09 2019). https://doi.org/10.1109/RE.2019.00025
11. Dalpiaz, F., Dell'Anna, D., Aydemir, F.B., Çevikol, S.: Requirements classification with interpretable machine learning and dependency parsing. In: RE'19. pp. 142–152. IEEE, Jeju, South Korea (2019)
12. De Martino, V., Castaño, J., Palomba, F., Franch, X., Martínez-Fernández, S.: A framework for using llms for repository mining studies in empirical software engineering. In: 2025 IEEE/ACM International Workshop on Methodological Issues with Empirical Studies in Software Engineering (WSESE). pp. 6–11. IEEE (2025)
13. De Martino, V., Zadenoori, M.A., Franch, X., Ferrari, A.: Green prompt engineering: Investigating the energy impact of prompt design in software engineering (2025)
14. Dekhtyar, A., Fong, V.: Re data challenge: Requirements identification with word2vec and tensorflow. In: 2017 IEEE 25th International Requirements Engineering Conference (RE). pp. 484–489. IEEE, Lisbon, Portugal (2017)
15. Eckhardt, J., Vogelsang, A., Fernández, D.M.: Are "non-functional" requirements really non-functional? an investigation of non-functional requirements in practice. In: Proceedings of the 38th international conference on software engineering. pp. 832–842 (2016)
16. Ernst, N.A., Mylopoulos, J.: On the perception of software quality requirements during the project lifecycle. In: Wieringa, R., Persson, A. (eds.) REFSQ. pp. 143–157. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
17. Ferrari, A., Dell'Orletta, F., Esuli, A., Gervasi, V., Gnesi, S.: Natural language requirements processing: A 4D vision. IEEE Softw. **34**(6), 28–35 (2017)
18. Hassani, S., Sabetzadeh, M., Amyot, D.: An empirical study on llm-based classification of requirements-related provisions in food-safety regulations. Empirical Software Engineering **30**(3), 72 (2025)
19. Hey, T., Keim, J., Koziolek, A., Tichy, W.F.: Norbert: Transfer learning for requirements classification. In: 2020 IEEE 28th International Requirements Engineering Conference (RE). pp. 169–179. IEEE, Zurich, Switzerland (2020)
20. Knauss, E., Houmb, S., Schneider, K., Islam, S., Jürjens, J.: Supporting requirements engineers in recognising security issues. In: Requirements Engineering: Foundation for Software Quality: 17th International Working Conference, (REFSQ 2011). pp. 4–18. Springer, Essen, Germany (March 2011)
21. Knauss, E., Houmb, S., Schneider, K., Islam, S., Jürjens, J.: Supporting requirements engineers in recognising security issues. In: REFSQ 2011. pp. 4–18. https://doi.org/$10.1007/978-3-642-19858-8_2$
22. Kurtanović, Z., Maalej, W.: Automatically classifying functional and non-functional requirements using supervised machine learning. In: 2017 IEEE 25th International Requirements Engineering Conference (RE). pp. 490–495. IEEE, Lisbon, Portugal (2017)
23. Olsson, T., Sentilles, S., Papatheocharous, E.: A systematic literature review of empirical research on quality requirements. Requirements Engineering **27**(2), 249–271 (Jun 2022). https://doi.org/10.1007/s00766-022-00373-9
24. Rejithkumar, G., Anish, P.R.: Nice: Non-functional requirements identification, classification, and explanation using small language models. In: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). pp. 284–295. IEEE (2025)

25. Wang, F., Zhang, Z., Zhang, X., Wu, Z., Mo, T., Lu, Q., Wang, W., Li, R., Xu, J., Tang, X., He, Q., Ma, Y., Huang, M., Wang, S.: A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness. ACM Trans. Intell. Syst. Technol. **16**(6) (Nov 2025). https://doi.org/10.1145/3768165, https://doi.org/10.1145/3768165
26. Winkler, J., Vogelsang, A.: Automatic classification of requirements based on convolutional neural networks. In: 2016 IEEE 24th International Requirements Engineering Conference Workshops (REW). pp. 39–45. IEEE, Beijing, China (2016)
27. Zadenoori, M.A., Dąbrowski, J., Alhoshan, W., Zhao, L., Ferrari, A.: Large language models (LLMs) for requirements engineering (RE): A systematic literature review (2025), https://arxiv.org/abs/2509.11446
28. Zadenoori, M.A., Zhao, L., Alhoshan, W., Ferrari, A.: Automatic prompt engineering: The case of requirements classification. In: International Working Conference on Requirements Engineering: Foundation for Software Quality. pp. 217–225. Springer (2025)
29. Zadenoori, M.A., Zhao, L., Alhoshan, W., Ferrari, A.: Automatic prompt engineering: The case of requirements classification. In: Hess, A., Susi, A. (eds.) REFSQ. pp. 217–225. Springer Nature Switzerland, Cham (2025)
30. Zhao, L., Alhoshan, W., Ferrari, A., Letsholo, K.J., Ajagbe, M.A., Chioasca, E.V., Batista-Navarro, R.T.: Natural language processing for requirements engineering: A systematic mapping study. ACM Comput. Surv. **54**(3) (Apr 2021)
31. Zhao, L., Alhoshan, W., Ferrari, A., Letsholo, K.J., Ajagbe, M.A., Chioasca, E.V., Batista-Navarro, R.T.: Natural language processing for requirements engineering: A systematic mapping study. ACM Computing Surveys (CSUR) **54**(3), 1–41 (2021)